

Client–Server Communication in Python using TCP, UDP, and Video Streaming

Sharique Ahmad

14-01-2024

1 Project 1: Socket Programming using TCP and UDP

1.1 Overview

The project demonstrates **client–server communication** using both **TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)**. The client sends a line of text to the server, and the server replies with the same text converted to **uppercase**. This shows the difference between connection–oriented (TCP) and connectionless (UDP) communication.

1.2 TCP Implementation

- The **server** creates a TCP socket, binds it to the address:

```
server_address = ('localhost', 12345)
```

and listens for incoming connections.

- When a client connects, the server receives data, converts it to uppercase, and sends it back.
- The **client** must use the same address and port:

```
server_address = ('localhost', 12345)
```

to connect, then it takes user input, sends it, and displays the modified response.

1.3 UDP Implementation

- The **server** creates a UDP socket bound to the address:

```
server_address = ('localhost', 54321)
```

and waits for datagrams.

- On receiving a message, it converts the text to uppercase and sends it back to the client.

- The **client** must use the same address and port:

```
server_address = ('localhost', 54321)
```

to send datagrams and then display the server's reply.

1.4 How to Run

1. Open two terminals.
2. In the first terminal, start the server:

```
python server_TCP.py
```

or

```
python server_UDP.py
```

3. In the second terminal, run the client:

```
python client_TCP.py
```

or

```
python client_UDP.py
```

4. Enter a line of text in the client. The server will return the same text in **uppercase**.

1.5 Working Principle

- In **TCP**, a reliable connection is established first, ensuring ordered delivery of data.
- In **UDP**, messages are sent as datagrams without a connection, making it faster but less reliable.
- Both implementations demonstrate the fundamental socket programming workflow:
 1. Create socket
 2. Bind/Connect
 3. Send/Receive
 4. Close connection

1.6 Troubleshooting (UDP Specific)

While running the UDP version on Windows, you may sometimes encounter:

```
ConnectionResetError: [WinError 10054] An existing connection was  
forcibly closed by the remote host
```

This happens when the client tries to receive a response but the server is not running or is bound to the wrong port. To fix this:

1. **Start the server first:** Run `python server_UDP.py`. The server should display UDP server is waiting for data....
2. **Run the client in another terminal:** Execute `python client_UDP.py`, then type a message (e.g. `hello`). The server will return the uppercase version (e.g. `HELLO`).
3. **Verify ports match:** Ensure both client and server use the same port. Example:

```
server_address = ('localhost', 54321)
```

must be identical in both files.

4. **Use IP address if needed:** On some Windows setups, replace `localhost` with `127.0.0.1` to avoid firewall/DNS issues.

Optionally, the client can be made more robust by wrapping `recvfrom()` in a try-except block:

```
try:
    modified_data, _ = client_socket.recvfrom(1024)
    print('Modified data:', modified_data.decode('utf-8'))
except ConnectionResetError:
    print("Error: UDP server not running or connection refused.")
```

1.7 Important Note on Compatibility

It is essential to understand that **TCP and UDP are two distinct protocols**. A **TCP server** can only communicate with a **TCP client**, and a **UDP server** can only communicate with a **UDP client**.

Attempting to run a **TCP server** while connecting with a **UDP client** (or vice versa) will not work, since the protocols operate in fundamentally different ways:

- TCP establishes a reliable, ordered connection.
- UDP sends standalone datagrams without connection.

If mismatched, the server and client cannot understand each other. The client may encounter errors such as:

```
ConnectionResetError: [WinError 10054]
```

Therefore, always ensure that the server and client **use the same protocol**. Only then does the communication flow smoothly.

2 Project 2: Real-Time Video Streaming using TCP

2.1 Overview

This project demonstrates **real-time video streaming** using **TCP socket programming** in Python. The **server** captures frames from a webcam using OpenCV, serializes the frames, and transmits them to a client over a TCP connection. The **client** receives the data, deserializes it, and displays the video in real time.

2.2 Server Implementation

- Creates a TCP socket bound to the address:

```
server_address = ('localhost', 5555)
```

- Accepts an incoming client connection.
- Captures video frames from the default webcam using OpenCV.
- Serializes the frames using `pickle`.
- Sends the frame size (using `struct.pack`) followed by the frame data.

2.3 Client Implementation

- Connects to the server at:

```
server_address = ('localhost', 5555)
```

- Receives the size of each incoming frame.
- Receives the corresponding frame data and deserializes it using `pickle`.
- Displays the video frames in a window using OpenCV.
- The video stream ends when the user presses `q`.

2.4 How to Run

1. Open two terminals.
2. In the first terminal, start the server:

```
python Server.py
```

3. In the second terminal, run the client:

```
python client.py
```

4. The client window will display the video captured by the server's webcam.
5. Press `q` in the client window to stop streaming.

2.5 Working Principle

- The **server** continuously reads webcam frames and sends them across the TCP connection.
- Each frame is preceded by its size, so the client knows how many bytes to read.
- The **client** reconstructs the frame and displays it in real time.
- This demonstrates how TCP ensures **ordered and reliable transmission**, which is crucial for video data.